TED UNIVERSITY

Department of Computer Engineering

MoonAI

Test Plan Report

Team:

Caner Aras Emir Irkılata Oğuzhan Özkaya

Supervisor: Ayşenur Birtürk

CMPE 492 — Senior Project II

Spring 2026

Table of Contents

1 Introduction **2**
1.1 Purpose . . . 2
1.2 Document Baseline . . . 2
1.3 Goals . . . 3

2 Test Items and Scope **3**
2.1 Primary Test Items . . . 3
2.2 In Scope . . . 4
2.3 Out of Scope . . . 4

3 Features To Be Tested **4**
3.1 Feature Groups . . . 5
3.2 Features Explicitly Not Covered by Current Tests . . . 6

4 Testing Methodology **6**
4.1 Unit Testing . . . 7
4.2 Integration Testing . . . 7
4.3 System Testing . . . 7
4.4 Performance Testing . . . 8
4.5 User Acceptance Testing . . . 12
4.6 Beta Testing . . . 12

5 Test Environment **12**
5.1 Software Environment . . . 13
5.2 Hardware and Platform Matrix . . . 13
5.3 Test Inputs and Expected Artifacts . . . 14

6 Test Cases **14**
6.1 Automated Unit Test Cases . . . 14
6.2 Integration and System Test Cases . . . 15
6.3 Performance, Acceptance, and Beta Test Cases . . . 16

7 Control Procedures and Schedule **17**
7.1 Entry Criteria . . . 17
7.2 Exit Criteria . . . 17
7.3 Failure Handling and Regression Control . . . 17
7.4 Continuous Integration and Code Quality Controls . . . 18
7.5 Test Schedule . . . 18

8 Roles, Risks, and Deliverables **19**
8.1 Roles and Responsibilities . . . 19
8.2 Risks and Mitigations . . . 19
8.3 Deliverables . . . 19

9 Glossary **20**

References **21**

1 Introduction

This document defines the current test plan for **MoonAI**, a modular predator-prey simulation platform used to study continuous neuroevolution with NEAT. The report is written against the repository state that exists today and describes testing assets, workflows, and verification activities that are implemented in the codebase, build system, or project infrastructure.

MoonAI contains several verification-relevant subsystems: C++ simulation and evolution libraries, an SFML-based visualization frontend, an optional CUDA acceleration path, command-line experiment orchestration, structured output logging, a Python analysis pipeline, a profiler executable, and GitHub Actions workflows for automated build and test execution. The test plan therefore has to cover more than unit-level algorithm checks; it must also describe end-to-end execution, artifact generation, performance observation, and manual user-oriented smoke testing.

1.1 Purpose

The purpose of this test plan is to:

- identify the MoonAI items currently under test,
- describe the features that are verified by automated and manual checks,
- define the test methodologies that are already implemented,
- record test control procedures, schedule, risks, and deliverables.

1.2 Document Baseline

The report is based on the following implemented project assets.

- **`tests/test_evolution.cpp`.** Provides automated GoogleTest coverage for NEAT data structures, mutation, crossover, speciation, JSON round-trip behavior, and neural network activation.
- **`tests/test_simulation_sensors.cpp`.** Provides automated GoogleTest coverage for simulation sensor encoding and density calculations.
- **`tests/CMakeLists.txt`.** Registers the test executable with GoogleTest discovery so the implemented unit tests can be executed through CTest.
- **`just test` / `ctest`.** Serve as the standard local entry point for automated regression execution.
- **`.github/workflows/ci.yml`.** Executes cross-platform automated build and test runs on Linux and Windows in Debug and Release configurations.
- **`.github/workflows/release.yml`.** Verifies release builds by running the automated test suite before packaging.

- **`src/main.cpp` and `src/app.cpp`.** Provide the runtime path used to verify configuration validation, CLI orchestration, headless or GUI execution, CPU or GPU selection, and output generation.
- **`analysis/moonai_analysis/`.** Implements the post-run analysis pipeline that converts experiment outputs into a self-contained HTML report.
- **`profiler/moonai_profiler/` and `src/profiler_main.cpp`.** Implement the built-in performance test and timing-report path for frame-time and scoped performance inspection.

1.3 Goals

The implemented testing approach focuses on the following goals:

- correctness of the highest-risk algorithmic logic in evolution and sensing,
- safe handling of configuration values and command-line overrides,
- stable end-to-end execution in GUI, headless, CPU-only, and GPU-enabled modes,
- integrity of generated output artifacts such as `config.json`, `stats.csv`, `species.csv`, and `genomes.json`,
- repeatable build and regression execution on both Linux and Windows,
- visibility into runtime performance through the profiler and Nsight tooling,
- internal user-oriented acceptance checks before accepting release-candidate builds.

2 Test Items and Scope

The system under test is not a single executable only; it is the complete MoonAI project distribution used during development, experimentation, profiling, and report generation.

2.1 Primary Test Items

- **Core configuration and CLI** (`src/core/`). Verification focuses on configuration serialization, validation rules, command-line parsing, and override handling.
- **Simulation engine** (`src/simulation/`). Verification focuses on agent sensing, state updates, headless execution, and interaction with evolution.
- **Evolution engine** (`src/evolution/`). Verification focuses on innovation tracking, mutation, crossover, species logic, and neural network execution.
- **Application runtime** (`src/app.*`). Verification focuses on startup, headless or GUI branching, CPU or GPU enablement, reporting cadence, and graceful shutdown.
- **Visualization** (`src/visualization/`). Verification focuses on manual smoke checks for rendering, controls, and agent inspection flows.

- **GPU acceleration path** (`src/gpu/`). Verification focuses on optional CUDA execution and CPU fallback behavior when GPU is disabled or unavailable.
- **Structured logging** (`src/core/logger.*`). Verification focuses on creation and format of run directories and machine-readable output files.
- **Analysis pipeline** (`analysis/moonai_analysis/`). Verification focuses on conversion of completed experiment outputs into a self-contained HTML report.
- **Profiler pipeline** (`profiler/moonai_profiler/`). Verification focuses on timed run capture, aggregation, and HTML profiling report generation.

2.2 In Scope

The following verification activities are in scope because they are already supported by the repository:

- automated unit tests registered through GoogleTest and CTest,
- automated cross-platform CI build-and-test execution,
- manual and scripted end-to-end runs of the main application,
- validation of command-line interfaces and configuration checking,
- verification of generated experiment, analysis, and profiler artifacts,
- internal manual acceptance and closed beta smoke checks by the three team members.

2.3 Out of Scope

The following items are explicitly out of scope for this report because no corresponding formal implementation exists in the current project:

- external public beta distribution or telemetry collection,
- cloud deployment testing,
- exhaustive CUDA hardware certification across multiple GPU models,
- security penetration testing,
- networked or distributed simulation testing,
- automated GUI interaction testing through a UI automation framework.

3 Features To Be Tested

This section lists the implemented product features that are covered by current testing activities. The emphasis is on features that already have executable verification paths, either through automated commands or through manual smoke procedures supported by the application.

3.1 Feature Groups

3.1.1 Configuration and Runtime

- **Config and CLI.** Validation of numeric bounds, handling of invalid overrides, experiment listing, experiment selection, runtime naming, headless mode, and CPU-only mode.
  Current verification mode: unit, integration, and manual testing.
- **Application startup.** Construction of simulation and evolution managers, optional visualization setup, signal handling, and final report logging.
  Current verification mode: integration and manual system testing.
- **Output generation.** Creation of run directory, configuration snapshot, metrics CSV, species CSV, genome JSON, and final flush at shutdown.
  Current verification mode: integration and manual artifact inspection.
- **GPU fallback path.** Runtime execution with optional GPU enablement and explicit `--no-gpu` fallback to CPU.
  Current verification mode: manual system and performance testing.

3.1.2 Core Algorithms

- **Evolution correctness.** Innovation numbering, mutation behavior, crossover behavior, species compatibility, network activation, and JSON round-trip behavior. Current verification mode: automated unit tests.
- **Simulation sensing.** Nearest-entity delta encoding, missing-target sentinels, and local density values in sensor vectors. Current verification mode: automated unit tests.

3.1.3 Tooling and Platform Support

- **Cross-platform build stability.** Build and test execution on Linux and Windows in Debug and Release configurations. Current verification mode: GitHub Actions CI.
- **Analysis reporting.** Discovery of completed runs and generation of a self-contained HTML analysis report. Current verification mode: integration testing.
- **Profiler reporting.** Capture of scoped timing data, generation of profiler JSON, and HTML performance report output. Current verification mode: performance testing workflow.

3.1.4 Visualization and Interaction

- **Visualization usability.** Launch, pause or resume, stepping, zoom, camera motion, agent selection, and graceful exit. Current verification mode: manual acceptance testing.

3.2 Features Explicitly Not Covered by Current Tests

Current tests do not attempt to prove the following because the supporting infrastructure is not present in the repository:

- usability studies with external non-team users,
- load balancing or distributed experiment execution,
- long-duration soak testing across the full 330-run matrix on every target platform,
- correctness claims for third-party dependencies beyond successful integration,
- GUI regression capture through screenshot-diff tooling.

4 Testing Methodology

MoonAI uses a layered testing approach. The implemented methodology starts with automated unit tests for algorithmic correctness, extends to integration and system-level command execution, and then adds performance observation and internal manual acceptance checks.

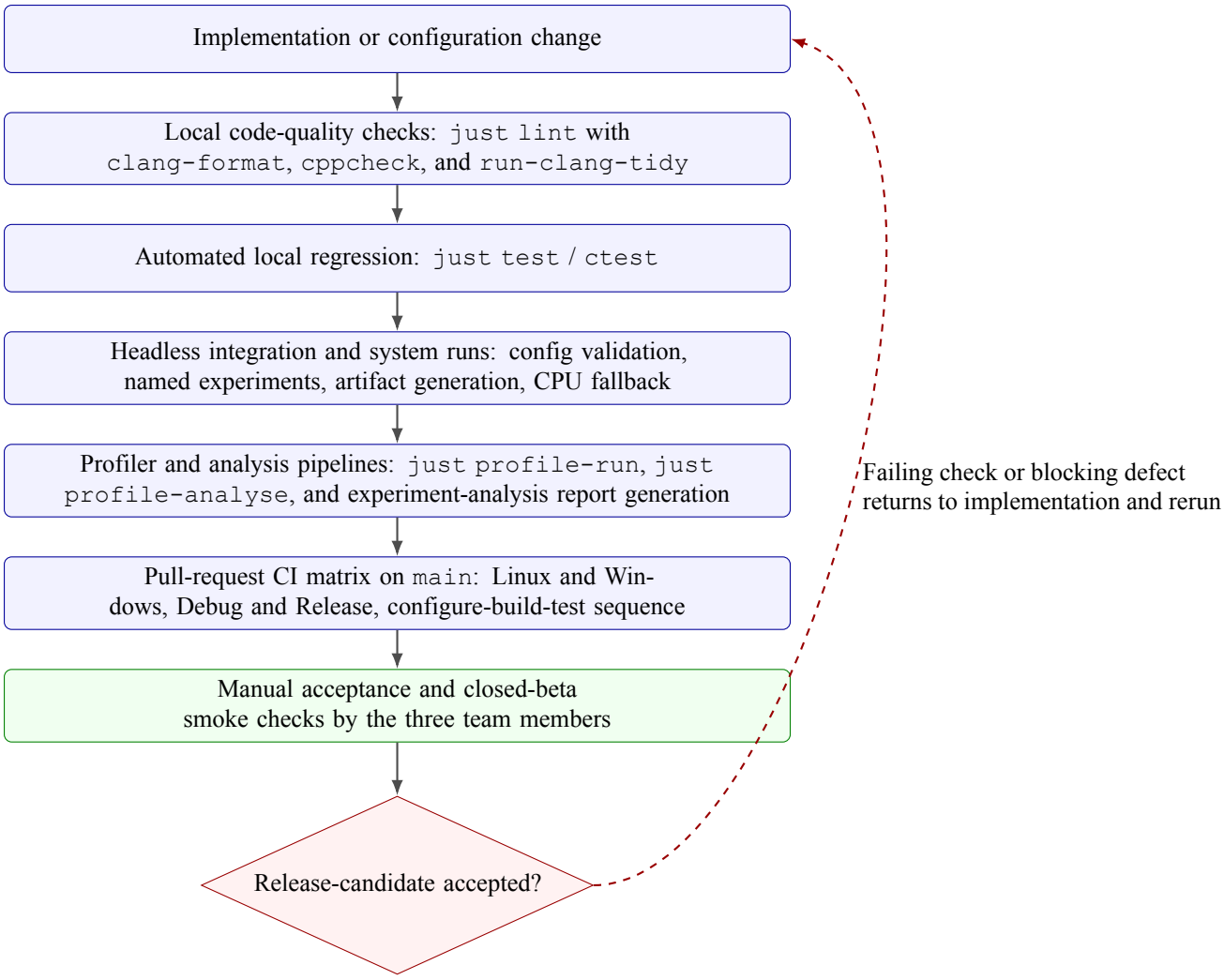


Figure 1: General validation-verification pipeline used by MoonAI during development and release-candidate testing.

Figure 1 summarizes the implemented verification flow from a source or configuration change to the final release-candidate decision. Automated checks dominate the earlier stages, while the final acceptance and beta pass remain manual.

4.1 Unit Testing

Automated unit testing is implemented with GoogleTest and registered through `gtest_discover_tests()` in `tests/CMakeLists.txt`. The current test suite covers the highest-risk deterministic logic:

- innovation number allocation and reset behavior,
- genome mutation and structural validity,
- crossover behavior and disabled-gene inheritance,
- species compatibility and complexity calculations,
- neural network activation and bounded outputs,
- simulation sensor vector encoding.

The standard local execution path is `just test`, which invokes CTest with `--output-on-failure`. The same test executable is also run in CI on Linux and Windows.

4.2 Integration Testing

Integration testing is performed by exercising implemented command chains that cross module boundaries. The most important integration flows are:

- config load and validation through `moonai --validate`,
- app startup from CLI arguments through `main.cpp` into `App`,
- headless execution that produces the structured output bundle,
- experiment analysis that converts run artifacts into HTML,
- profiler execution that produces timing JSON and profiler HTML output.

These tests verify that independently correct subsystems work together with the actual file formats and runtime wiring used by the project.

4.3 System Testing

System testing treats MoonAI as a finished executable. The current system scenarios include:

- GUI launch on a machine with a display server,
- headless launch on a machine without visualization,
- CPU-only execution through `--no-gpu`,
- experiment discovery via `--list` and selection via `--experiment`,
- generation of the output directory and final report records.

Because the project uses SFML for visualization, the GUI part of system testing is manual. Automated CI covers the build-and-test portion but not UI interaction.

4.4 Performance Testing

Performance testing is implemented through the dedicated profiler executable and helper commands in the project `justfile`. The profiler captures per-frame timing trees, aggregates repeated runs, and produces a self-contained HTML report. The current performance-oriented workflow includes:

- release-build profiler runs through `just profile-run`,
- profiler report generation through `just profile-analyse`,
- full profiler pipeline through `just profile`,
- optional GPU kernel inspection with `just ncu`,
- optional system-level CUDA tracing with `just nsys`.

The collected metrics include average frame time, variability across repeated runs, and timing distribution across major scoped events.

4.4.1 Implemented Profiler Execution

The dedicated profiler executable is implemented in `src/profiler_main.cpp`. It runs a fixed multi-seed profiling suite and writes one JSON manifest per suite.

- **Default suite shape.** The current defaults are 600 sampled frames, speed multiplier 64, seeds 61 through 66, output directory `output/profiles`, and suite name `profile`.
- **Supported arguments.** The executable currently accepts `--frames`, `--output-dir`, `--name`, and `--no-gpu`.
- **Collected data.** Each run records a per-frame scope tree, total runtime, average frame time, and the seed associated with the run.
- **Aggregation rule.** Completed runs are sorted by average frame time. When more than two runs finish successfully, the fastest and slowest runs are marked as dropped and the remaining middle runs are kept for aggregate reporting.
- **Written manifest fields.** The generated JSON manifest contains suite metadata, per-run dispositions, summary statistics such as average and standard deviation, an averaged hierarchical scope tree, and a frame timeline in milliseconds.

4.4.2 Profiler Analysis Pipeline

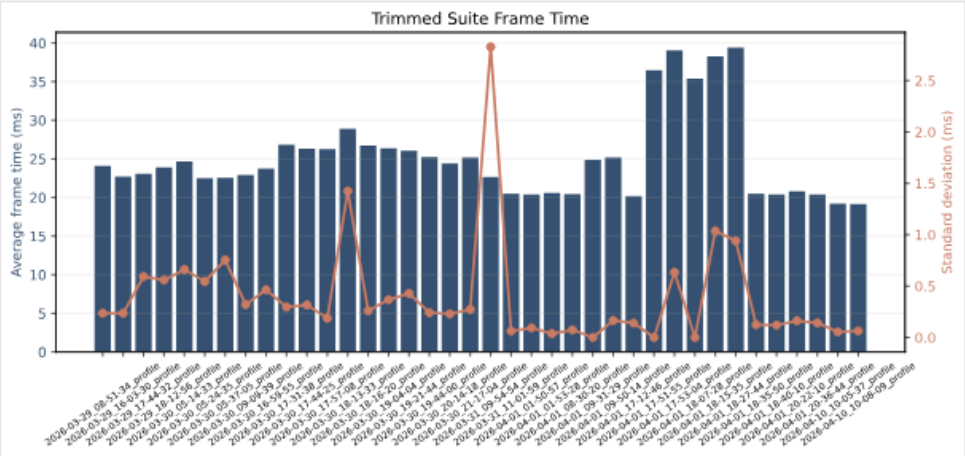
The Python-side profiler analysis is implemented under `profiler/moonai_profiler/`. Its command-line entry point is `__main__.py`, which by default reads profiler manifests from `output/profiles/` and writes HTML reports to `profiler/output/`.

- **Input loading.** `io.py` parses the manifest JSON files into structured dataclasses such as `ProfileSuite`, `SuiteMember`, and `AveragedNode`.
- **Report generation.** `report.py` builds summary rows, frame-time comparison charts, per-suite timeline charts, per-suite member charts, hierarchical event rows, and optional flame-graph visualizations.
- **HTML rendering.** `html_report.py` renders the final self-contained report through the Jinja template at `profiler/moonai_profiler/templates/report.html.j2`.
- **Generated outputs.** The analysis step writes a timestamped HTML file such as `profile_report_YYYYMMDD_HHMMSS.html` and also refreshes `profiler/output/profile.html` as a stable latest-report path.

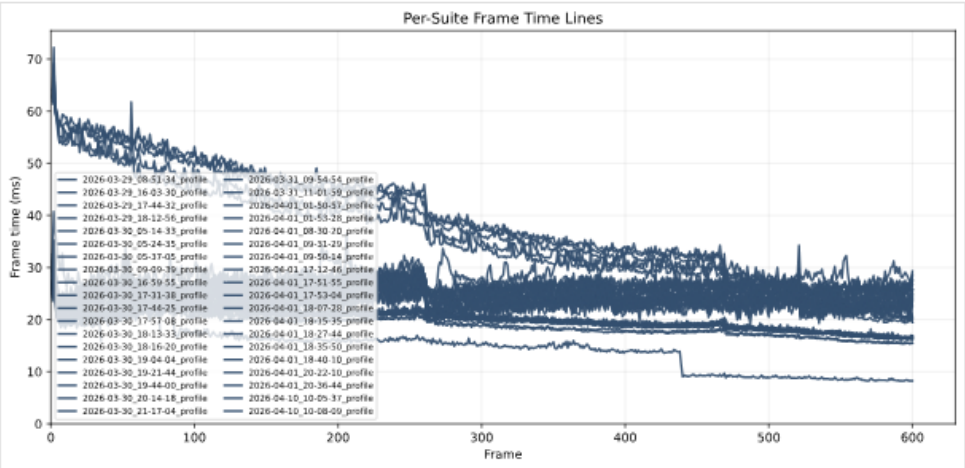
The generated profiler report exposes both suite-level summaries and detailed run-level timing views. The screenshots below illustrate the two most important report perspectives currently used during performance evaluation.

2026-04-01_17-53-04_profile	600	4	39.05	+7.0%
2026-04-01_18-07-28_profile	600	1	35.38	-9.4%
2026-04-01_18-15-35_profile	600	4	38.28	+8.2%
2026-04-01_18-27-44_profile	600	4	39.41	+2.9%
2026-04-01_18-35-50_profile	600	4	20.51	-48.0%
2026-04-01_18-40-10_profile	600	4	20.40	-0.5%
2026-04-01_20-22-10_profile	600	4	20.78	+1.8%
2026-04-01_20-36-44_profile	600	4	20.37	-1.9%
2026-04-10_10-05-37_profile	600	4	19.20	-5.7%
2026-04-10_10-08-09_profile	600	4	19.13	-0.4%

Cross-Run Comparisons



Average Frame Time. Bars: Mean of middle four runs. Line: Standard deviation (lower is more stable).



Per-Suite Frame Time Lines. Per-frame mean across suite seeds, using kept runs when available.

Figure 2: Profiler suite overview showing run-summary information and cross-run frame-time comparison charts. This view is used first to compare overall timing behavior across kept runs and suites.

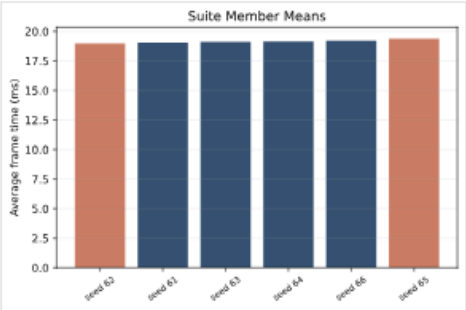
▾ **2026-04-10_10-08-09_profile**

Metadata

SUITE NAME	FRAME COUNT	REPORT INTERVAL	PLATFORM
profile	600	N/A steps	linux
GPU ALLOWED	**CUDA**	**OPENMP**	**GENERATED**
Yes	Yes	No	2026-04-10_10-09-19

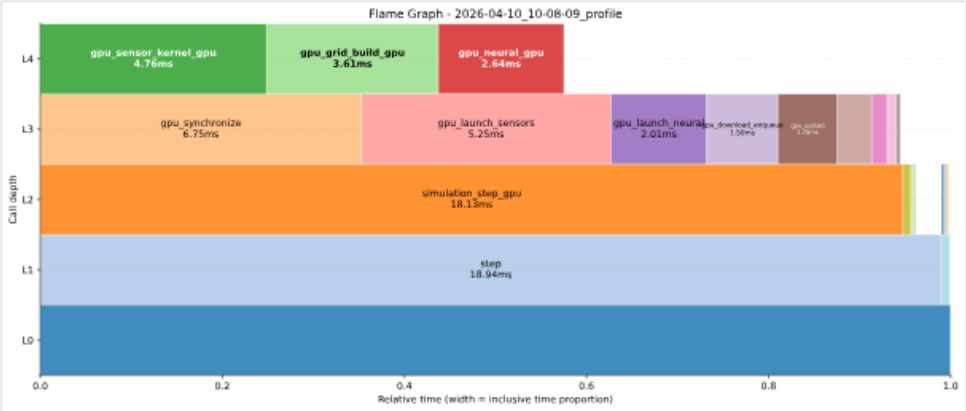
Suite Members

Seed	Avg Frame (ms)	Disposition
62	18.98	dropped_fastest
61	19.05	kept
63	19.12	kept
64	19.15	kept
66	19.22	kept
65	19.37	dropped_slowest



Member Run Means. Orange bars are dropped runs. Blue bars are kept runs.

Flame Graph



Flame Graph. Call hierarchy visualization. Width = inclusive time proportion, height = call depth. Hover over blocks for details.

Events

Event	%	Avg (ms)	Count	Total (ms)
frame_total	100.0	19.134	2400	45922.466
step	99.0	18.942	153600	45459.916
simulation_step_gpu	95.7	18.133	153600	43519.432
gpu_synchronize	37.2	6.749	153600	16197.393
gpu_sensor_kernel_gpu	70.5	4.757	153600	11416.093
gpu_grid_build_gpu	53.5	3.610	153600	8663.598
gpu_neural_gpu	39.1	2.640	153600	6335.319
gpu_launch_sensors	[illegible]	[illegible]	[illegible]	[illegible]

Figure 3: Detailed profiler report view showing hierarchical timing breakdowns, including the event table and flame-graph style visualization used to inspect scoped runtime costs inside a kept suite.

4.5 User Acceptance Testing

Formal external acceptance testing is not implemented in the repository. The current user acceptance process is therefore internal and manual. The participants are:

- Oğuzhan Özkaya,
- Emir Irkılata,
- Caner Aras.

Each participant executes a short set of end-user scenarios on the built application: start the simulation, interact with the visualization controls, run a headless experiment, inspect generated outputs, and confirm that the analysis or profiler reports open correctly.

4.6 Beta Testing

MoonAI does not currently have an external public beta program, public telemetry, or a standalone bug-reporting portal. The implemented beta process is therefore a **closed internal beta smoke pass** performed by the same three team members on a release-candidate build. This internal beta focuses on regression detection and platform sanity rather than large-scale external feedback.

5 Test Environment

The current MoonAI test environment combines local developer machines and hosted CI executors. The project supports both CPU-only and optional CUDA-enabled execution.

5.1 Software Environment

Table 1: Software environment used by the implemented tests

Category	Tool	Role
Build system	CMake 3.21+	Configures the project, test targets, optional CUDA support, and profiler build.
Build runner	Ninja	Recommended local build backend for fast incremental builds.
Dependency manager	vcpkg	Supplies SFML, GoogleTest, sol2, spdlog, and JSON dependencies used in tests and application builds.
Unit test framework	GoogleTest	Executes C++ unit tests for simulation and evolution logic.
Test launcher	CTest	Standard command-line interface for running discovered tests locally and in CI.
Automation entry point	just	Wraps common build, test, profiling, and analysis commands.
Visualization runtime	SFML 3	Provides the GUI surface used in manual system and acceptance testing.
Analysis runtime	Python + uv	Runs the analysis and profiler HTML report generators.
CI platform	GitHub Actions	Executes cross-platform build and test workflows.
Performance tooling	moonai_profiler, ncu, nsys	Measures frame timing and optional GPU kernel behavior.

5.2 Hardware and Platform Matrix

Current platform coverage is summarized below.

- **Linux developer machine.** Main local environment for building, running, profiling, report generation, and GUI validation.
- **Windows GitHub Actions runner.** Automated CI coverage for build and test execution in Debug and Release configurations.
- **Linux GitHub Actions runner.** Automated CI coverage for build and test execution in Debug and Release configurations.
- **CUDA-capable Linux machine.** Manual performance and system coverage for the optional GPU path, profiler GPU timing, Nsight Compute, and Nsight Systems workflows.
- **Headless environment.** Non-visual execution mode used for experiment runs, artifact generation, and report pipelines.

5.3 Test Inputs and Expected Artifacts

The primary runtime input is `config.lua`, optionally combined with CLI flags such as `--experiment`, `--headless`, `--validate`, `--set`, and `--no-gpu`. Successful test execution may generate the following artifacts:

- `config.json` snapshot for the executed run,
- `stats.csv` time-series metrics,
- `species.csv` species-level summaries,
- `genomes.json` representative genome snapshots,
- analysis HTML reports under `analysis/output/`,
- profiler suite manifests under `output/profiles/`,
- profiler HTML reports under `profiler/output/`, including the stable latest-copy path `profile.html`,
- CI logs for automated build-and-test validation.

6 Test Cases

Each test case below is assigned a stable test identifier. The cases are derived directly from implemented repository capabilities and therefore map to existing automated tests, supported commands, or manual smoke procedures.

6.1 Automated Unit Test Cases

- **TP-UT-001 Innovation tracking consistency.**
  Owner: Oğuzhan Özkaya.
  Procedure: Execute the same-connection innovation-tracking case from `test_evolution.cpp` through CTest.
  Expected result: Repeated requests for the same connection pair return the same innovation identifier within the same mutation window.

- **TP-UT-002 Mutation preserves executable genomes.**
  Owner: Oğuzhan Özkaya.
  Procedure: Execute the mutation-validity case from `test_evolution.cpp`.
  Expected result: Repeated mutation still produces a genome that can be converted into a neural network and activated with bounded outputs.

- **TP-UT-003 Crossover produces valid offspring.**
  Owner: Oğuzhan Özkaya.
  Procedure: Execute the crossover-validity case from `test_evolution.cpp`.
  Expected result: The child genome remains structurally valid and produces the expected output count when activated.

- **TP-UT-004 Sensor vector encoding correctness.**

Owner: Emir Irkılata.
Procedure: Execute the sensor-encoding case from `test_simulation_sensors.cpp`.
Expected result: Predator and prey sensor vectors encode the correct nearest-target deltas, sentinel values, and density metrics.

6.2 Integration and System Test Cases

- **TP-IT-001 Config validation and override rejection.**
  Owner: Caner Aras.
  Procedure: Run `just validate` and repeat validation with intentionally invalid `--set` values.
  Expected result: Valid configurations print `OK`; invalid values report field-specific errors and stop execution.
- **TP-IT-002 Headless run artifact generation.**
  Owner: Caner Aras.
  Procedure: Run `just run -- --headless --steps 60` and inspect the created output directory.
  Expected result: The application completes headlessly and writes `config.json`, `stats.csv`, `species.csv`, and `genomes.json`.
- **TP-IT-003 Analysis pipeline integration.**
  Owner: Caner Aras.
  Procedure: After a completed headless run, execute `just experiment-analyse`.
  Expected result: A self-contained HTML analysis report is generated under `analysis/output/`.
- **TP-IT-004 Profiler pipeline integration.**
  Owner: Caner Aras.
  Procedure: Build the profiler path and execute `just profile-run`; then execute `just profile-analyse`.
  Expected result: Profiler JSON is written under `output/profiles/` and an HTML report is generated under `profiler/output/`.
- **TP-ST-001 GUI startup and shutdown.**
  Owner: Emir Irkılata.
  Procedure: Launch `just run` on a machine with a display server, observe startup, then close the window.
  Expected result: The application opens successfully, renders the simulation, and exits cleanly while flushing final output files.
- **TP-ST-002 CPU fallback path.**
  Owner: Emir Irkılata.
  Procedure: Execute `just run -- --headless --no-gpu --steps 60`.

Expected result: The simulation runs successfully without CUDA and still produces the standard output bundle.

- **TP-ST-003 Experiment discovery and selection.**
  Owner: Caner Aras.
  Procedure: Execute `just list-experiments`; then run one named experiment with `--experiment`.
  Expected result: The executable lists known experiments and runs the selected one without manual config edits.

- **TP-ST-004 Cross-platform CI regression.**
  Owner: Caner Aras.
  Procedure: Trigger or inspect the implemented CI workflows for Linux and Windows Debug and Release jobs.
  Expected result: Each job configures, builds, and runs the test suite without failure before merge or release packaging.

6.3 Performance, Acceptance, and Beta Test Cases

- **TP-PT-001 Release-build frame-time profiling.**
  Owner: Oğuzhan Özkaya.
  Procedure: Execute `just profile-run` on a release build and inspect the generated summary report.
  Expected result: Average frame time, kept-run counts, and timing charts are produced successfully for the sampled profiler suite.

- **TP-PT-002 GPU performance inspection.**
  Owner: Oğuzhan Özkaya.
  Procedure: On a CUDA-capable Linux machine, execute `just ncu` or `just nsys`.
  Expected result: GPU profiling completes and reports timing information for the targeted kernel or traced run.

- **TP-UAT-001 Internal GUI usability smoke pass.**
  Owner: Emir Irkılata.
  Procedure: Oğuzhan Özkaya, Emir Irkılata, and Caner Aras each run the GUI build, pause or resume, step once, zoom, pan, and select an agent.
  Expected result: Core controls respond correctly and the displayed simulation state remains usable for inspection.

- **TP-UAT-002 Internal experiment workflow acceptance.**
  Owner: Caner Aras.
  Procedure: The same three participants execute a named headless experiment, inspect the output files, and open the generated analysis report.
  Expected result: A project user can complete the standard experiment workflow without

changing the source code.

- **TP-BT-001 Closed internal beta smoke pass.**
  Owner: Emir Irkılata.
  Procedure: Before final submission, the team repeats the core system, analysis, and performance-related smoke checks on the release-candidate state.
  Expected result: No blocking regression remains in build, test, runtime, profiling, or core experiment workflows.

7 Control Procedures and Schedule

This section defines how test execution is controlled, when it is considered complete, and how the work is scheduled relative to the report deadline.

7.1 Entry Criteria

Testing may begin when all of the following are true:

- the project configures successfully with CMake,
- required dependencies are available through vcpkg,
- the selected build includes `MOONAI_BUILD_TESTS=ON`,
- the chosen configuration file passes basic syntax loading,
- the intended execution mode is available: display server for GUI tests or headless mode for non-visual tests,
- the team agrees on the test build or release-candidate revision being exercised.

7.2 Exit Criteria

Testing is considered complete for a report submission when the following conditions hold:

- the automated CTest suite completes without failure,
- CI jobs for supported platforms remain green,
- manual system and acceptance smoke cases are completed by the designated team members,
- required runtime artifacts are generated successfully.

7.3 Failure Handling and Regression Control

- a local or CI test failure blocks the corresponding change from being considered complete,
- fixes are followed by rerunning the failed automated command or repeating the failed manual scenario,
- release packaging is gated by automated tests in the existing release workflow.

7.4 Continuous Integration and Code Quality Controls

MoonAI includes concrete automated build-and-test workflows together with a developer-side code-quality command path.

- **Pull-request CI coverage.** The implemented CI workflow in `.github/workflows/ci.yml` runs on pull requests targeting `main`. It builds and tests the project on Linux and Windows, and for each platform it exercises both Debug and Release configurations.
- **Release verification coverage.** The implemented release workflow in `.github/workflows/release.yml` runs on version tags matching `v*.*.*`. It rebuilds release binaries for Linux and Windows, reruns the test suite, and only then packages the release artifacts.
- **Merge-control note.** The repository source clearly defines CI status checks for pull requests to `main`. Hard enforcement of the rule that failed checks cannot be merged into `main` is controlled by GitHub branch-protection settings, which are configured in the repository settings rather than stored in the source tree. In the intended workflow, failed CI checks are treated as merge blockers for main-branch changes.
- **Formatter and linter setup.** The developer command `just lint` performs three implemented code-quality steps: automatic source formatting with `clang-format --style=file -i`, static analysis with `cppcheck`, and deeper compilation-aware analysis with `run-clang-tidy` against the generated `compile_commands.json` database.

7.5 Test Schedule

The schedule below is expressed over the first eight working weeks of the semester.

1. **Weeks 1–2.** Run and stabilize automated unit tests, inspect early CI failures, and verify configuration validation paths.
2. **Weeks 3–4.** Execute headless integration scenarios, artifact checks, and cross-platform runtime sanity checks.
3. **Week 5.** Run release-build profiler suites and optional GPU performance checks on the available CUDA machine.
4. **Week 6.** Review profiler-analysis outputs, compare timing summaries, and confirm that the reporting pipeline produces usable HTML artifacts.
5. **Week 7.** Perform internal user-acceptance and closed-beta smoke checks with Oğuzhan Özkaya, Emir Irkılata, and Caner Aras.
6. **Week 8.** Consolidate test evidence, review remaining defects, and finalize the release-candidate verification record.

8 Roles, Risks, and Deliverables

8.1 Roles and Responsibilities

The project team shares responsibility for implementing, running, and reviewing the current MoonAI test process. Because this is a course project with a small team, responsibilities are intentionally lightweight and collaborative.

Table 2: Testing roles and responsibilities

Responsible personnel	Testing responsibility
Oğuzhan Özkaya	Owns evolution-focused automated tests together with profiler and GPU-oriented performance verification.
Emir Irkılata	Owns simulation, visualization, CPU-fallback, and manual acceptance or beta smoke verification.
Caner Aras	Owns configuration, integration flows, analysis-pipeline checks, and cross-platform CI regression verification.

8.2 Risks and Mitigations

Table 3: Key testing risks

Risk	Impact	Mitigation
CUDA hardware unavailable	GPU-specific performance and runtime checks cannot be executed on every machine.	Keep CPU fallback test coverage active and run GPU checks only on the available CUDA-capable machine.
GUI requires display access	GUI smoke tests cannot run inside fully headless automation.	Keep GUI checks manual and rely on headless automated paths for CI.
Cross-platform dependency drift	Linux and Windows builds may diverge because of toolchain or dependency changes.	Continue running both platforms in CI for pull requests and release workflows.
Large experiment runtime cost	Full-scale performance or experiment runs may exceed available time before the deadline.	Use shorter smoke runs for test verification and reserve long runs for analysis tasks when time permits.
Small manual tester pool	Internal acceptance and beta coverage are limited to three team members.	Use a repeatable smoke checklist and ensure all three team members run the same scenarios.

8.3 Deliverables

The implemented test process produces the following deliverables:

- automated test evidence through local CTest runs and GitHub Actions logs,
- generated runtime artifacts for experiment, analysis, and profiler flows.

9 Glossary

CI Continuous Integration. In MoonAI this refers to the GitHub Actions workflows that configure, build, and test the project automatically on Linux and Windows.

CTest CMake's command-line test runner. MoonAI uses it to execute GoogleTest-discovered unit tests locally and in CI.

CUDA NVIDIA parallel-computing platform used by MoonAI for optional GPU acceleration.

Headless mode Execution mode enabled by `--headless`. The simulation runs without creating the SFML visualization window.

Internal beta The closed release-candidate smoke pass executed only by the project team, not by external public users.

NEAT NeuroEvolution of Augmenting Topologies, the evolutionary algorithm used by MoonAI for evolving neural network topology and weights.

Nsight Compute NVIDIA profiling tool invoked by the `just ncu` recipe for kernel-level GPU performance inspection.

Nsight Systems NVIDIA profiling tool invoked by the `just nsys` recipe for CUDA and runtime tracing.

Profiler suite A repeated set of timed runs collected by `moonai_profiler` and summarized in an HTML report.

Smoke test A short verification run intended to prove that a major workflow still works after recent changes.

UAT User Acceptance Testing. In this project it is implemented as an internal manual check by Oğuzhan Özkaya, Emir Irkılata, and Caner Aras.

References

[1] Google LLC, *GoogleTest – Google Testing and Mocking Framework*, 2024. [Online]. Available: https://github.com/google/googletest

[2] Kitware Inc., *CMake Build System*, version 3.21+, 2024. [Online]. Available: https://cmake.org

[3] NVIDIA Corporation, *CUDA C++ Programming Guide*, version 12.x, 2024. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/

[4] GitHub, *GitHub Actions Documentation*, 2024. [Online]. Available: https://docs.github.com/actions

[5] Google LLC, *GoogleTest Primer*, 2024. [Online]. Available: https://google.github.io/googletest/primer.html

[6] C. Casey, *just – a command runner*, 2024. [Online]. Available: https://github.com/casey/just

[7] SFML Development Team, *SFML – Simple and Fast Multimedia Library*, version 3, 2024. [Online]. Available: https://www.sfml-dev.org

[8] ISO/IEC/IEEE, *ISO/IEC/IEEE 29119-3 – Software and Systems Engineering – Software Testing – Part 3: Test Documentation*, 2021.